Reviewer Pack — Minimal Geometric Entropy of Tensor Products and Circuit Dep…

Entry a5e21c5ba3e4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 11:33:00 UTC

Minimal Geometric Entropy of Tensor Products and Circuit Depth

Entry ID: `a5e21c5ba3e4` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 11:33:00 UTC

1. Conjecture

Field A (mathematical branch): Information Geometry **Field B** (complexity object): Boolean Circuits (Circuit Depth)

Statement:

> For any Boolean function f with m inputs, the geometric entropy H(f) of its tensor product representation, when computed using the Fisher-Rao metric on the manifold of probability distributions over the input space, is upper-bounded by O(m log n), where n is the circuit depth of f.

Rationale (proposer's reasoning):

> Information geometry provides a framework to study the geometry of probability distributions, and its geometric entropy can capture the complexity of these distributions. By linking this concept with Boolean circuits, we may expose a new perspective on circuit complexity. The tensor product representation of Boolean functions allows for the study of their joint behavior, which could reveal deeper connections between information geometry and computational complexity.

Taxonomy category: `information_geometry_circuit_depth` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5b3b6f6c6641ebfa`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 80% of the randomly generated m-input Boolean functions f, the geometric entropy H(f ⊗ f) calculated using the Fisher-Rao metric is less than or equal to O(m log n), where n is the circuit depth of f. The conjecture is falsified if any seed produces a geometric entropy greater than O(m log n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tensor product" AND "geometric entropy" AND "Boolean circuits" AND "circuit depth" - "Fisher-Rao metric" AND "manifold of probability distributions" AND "Boolean function" AND "information geometry" - "upper bound" O(m log n) AND "geometric entropy" AND "tensor product" AND "Boolean circuit depth"

Top relevant hits considered: - [http://arxiv.org/abs/0901.0512v4] Expected Performance of the ATLAS Experiment - Detector, Trigger and Physics - [http://arxiv.org/abs/2303.17007v2] Impact of cross-section uncertainties on supernova neutrino spectral parameter fitting in the Deep Underground Neutrino - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \to \mu^+\mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(m):
        return [random.choice([0, 1]) for _ in range(2**m)]

    def tensor_product(f, g):
        m = len(f)
        n = len(g)
        result = []
        for i in range(2**(m+n)):
            x = bin(i)[2:].zfill(m+n)
            x_m = int(x[:m], 2)
            x_n = int(x[m:], 2)
            result.append(f[x_m] * g[x_n])
        return result

    def geometric_entropy(p):
        p = [x for x in p if x > 0]
        H = -sum([p_i * math.log2(p_i) for p_i in p]) / len(p)
```

```
        return H

    def circuit_depth(f):
        m = len(f)
        n = 1
        while True:
            new_f = [f[i] ^ f[(i + 1) % m] for i in range(m)]
            if all(new_f):
                break
            f = new_f
            n += 1
        return n

    def fisher_rao_metric(p, q):
        p = [x for x in p if x > 0]
        q = [x for x in q if x > 0]
        if len(p) != len(q):
            return float('inf')
        H_p = geometric_entropy(p)
        H_q = geometric_entropy(q)
        I_pq = sum([p_i * math.log2(p_i / q_i) for p_i, q_i in zip(p, q)]) / len(p)
        return 2 * (H_p + H_q - 2 * I_pq)

    m = random.randint(5, 30)
    f = generate_boolean_function(m)
    g = tensor_product(f, f)
    n = circuit_depth(f)
    H_g = geometric_entropy(g)
    O_m_log_n = m * math.log(n)

    return {
        "metric_name": "geometric_entropy",
        "metric_value": H_g,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": H_g <= O_m_log_n,
        "counterexample": "" if H_g <= O_m_log_n else f"m={m}, n={n}, H(g)={H_g}, O(m log n)={O_m_
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
```

```
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14742
2	preregistration	ollama_remote	glm4:latest	0	0	9337
3	novelty	ollama_remote	glm4:latest	0	0	13818
4	novelty	ollama_remote	glm4:latest	0	0	9262
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17600
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9306
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7337
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30179
9	judge	ollama_remote	glm4:latest	0	0	45051

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 156633 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a5e21c5ba3e4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/a5e21c5ba3e4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a5e21c5ba3e4.tar.gz` (if generated)